

Final Report

Team: Benigno

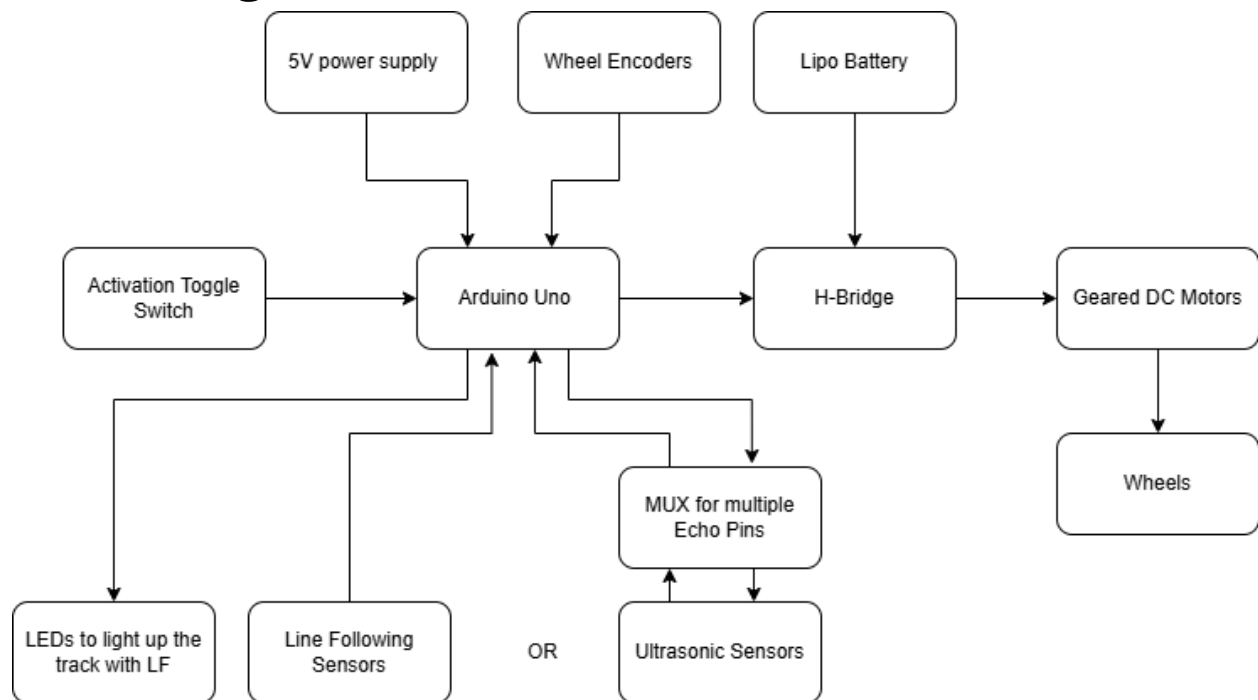
Team Members: Diego Morales, Jarren Hill, Kevyn Vital

Introduction

This report outlines the development of a smart car, designed to autonomously navigate through a maze using ultrasonic sensors and follow a line using line followers and wheel encoders. The project was built on an Arduino Uno with DC motors controlled by an H-Bridge and PWM signals. We describe the hardware components, I/O pin assignments, and circuit setups used in both the barrier detection and line following demos. The software section explains the logic behind the motor control, input sensors, and ISR's. Functionality, challenges, design changes, and attempted feedback control are discussed, along with our two enhancements the LCD odometer and three-point turn routine.

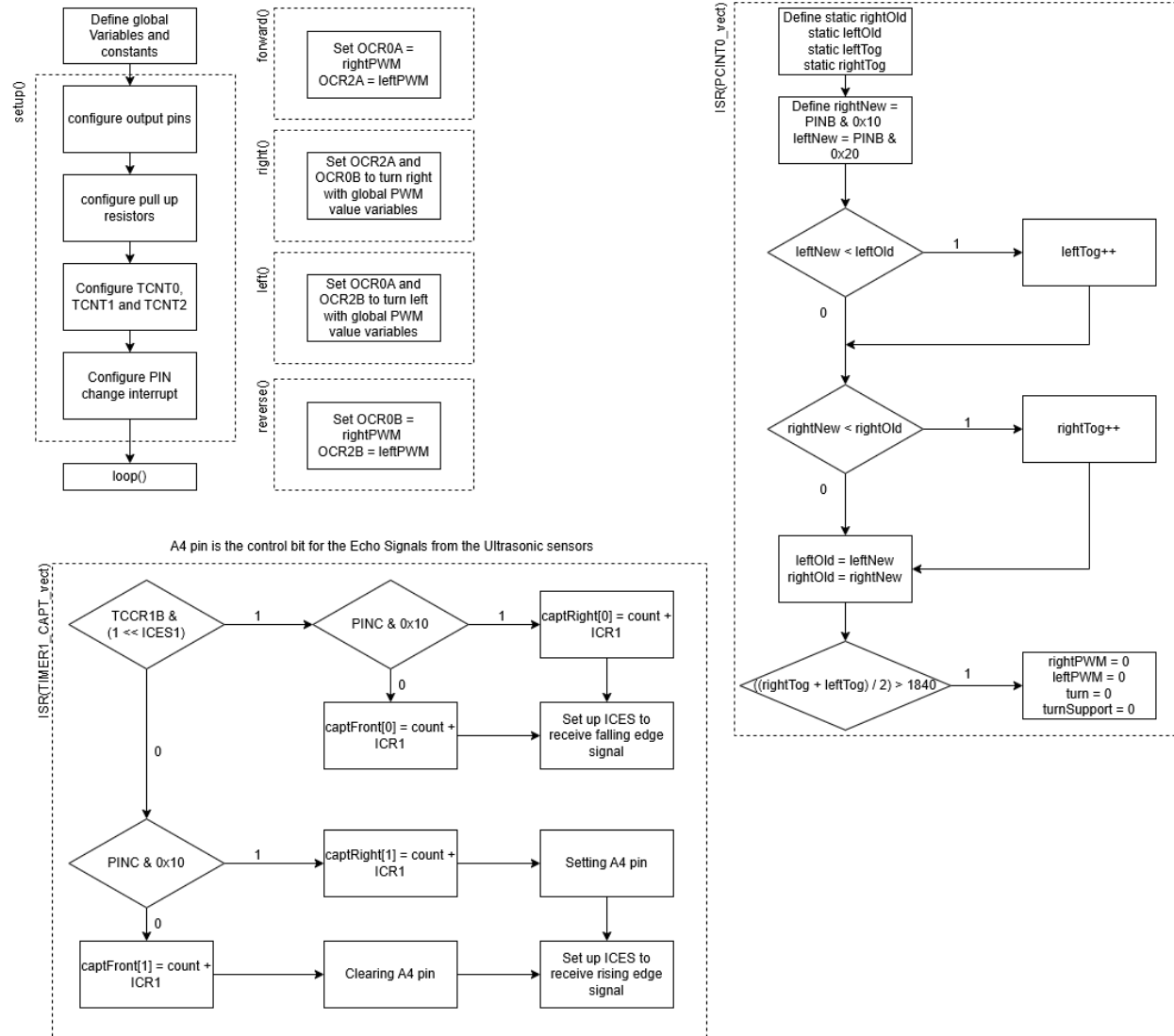
Each section provides a detailed breakdown of how the car operates and the process behind our design decisions.

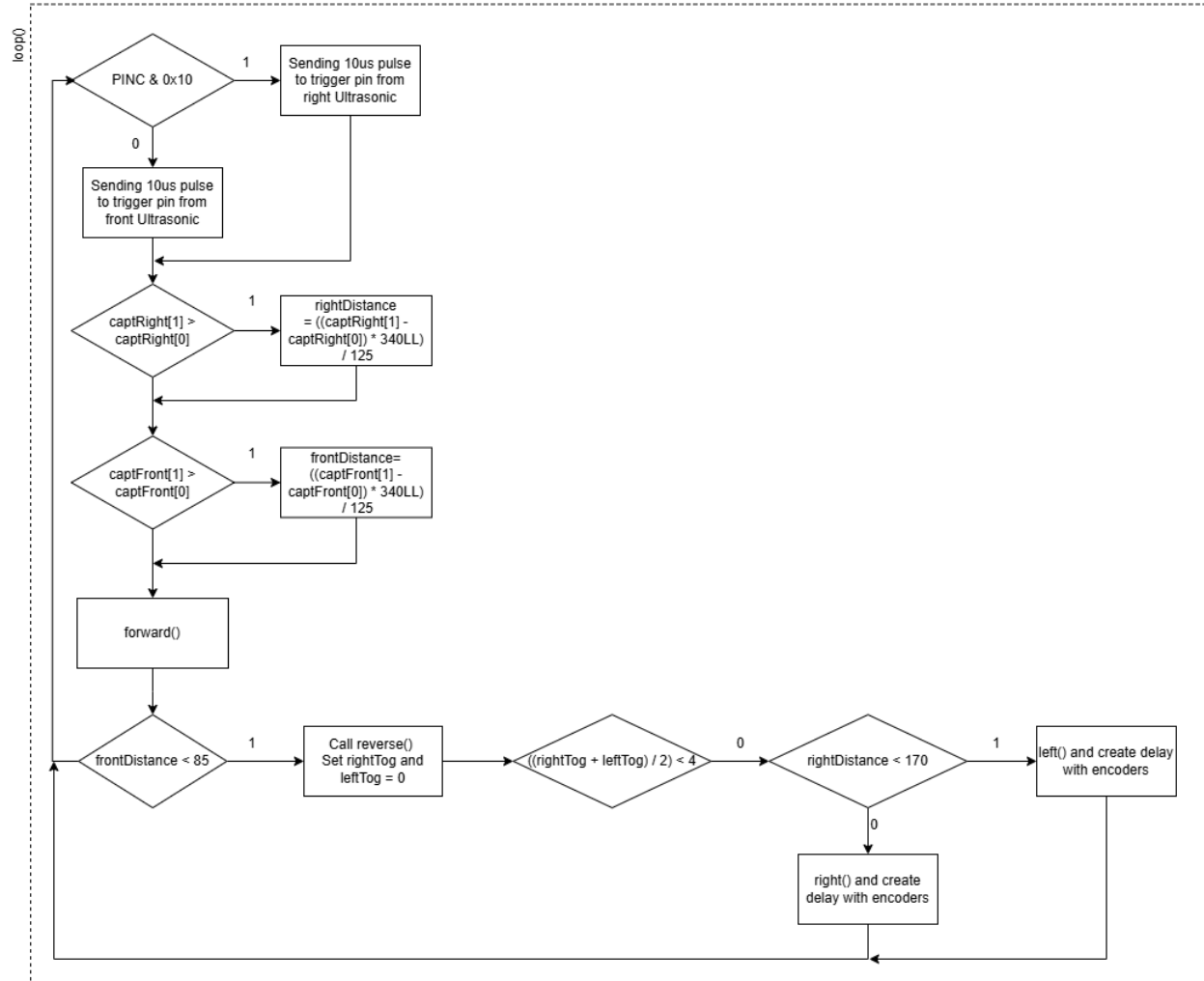
Block Diagram



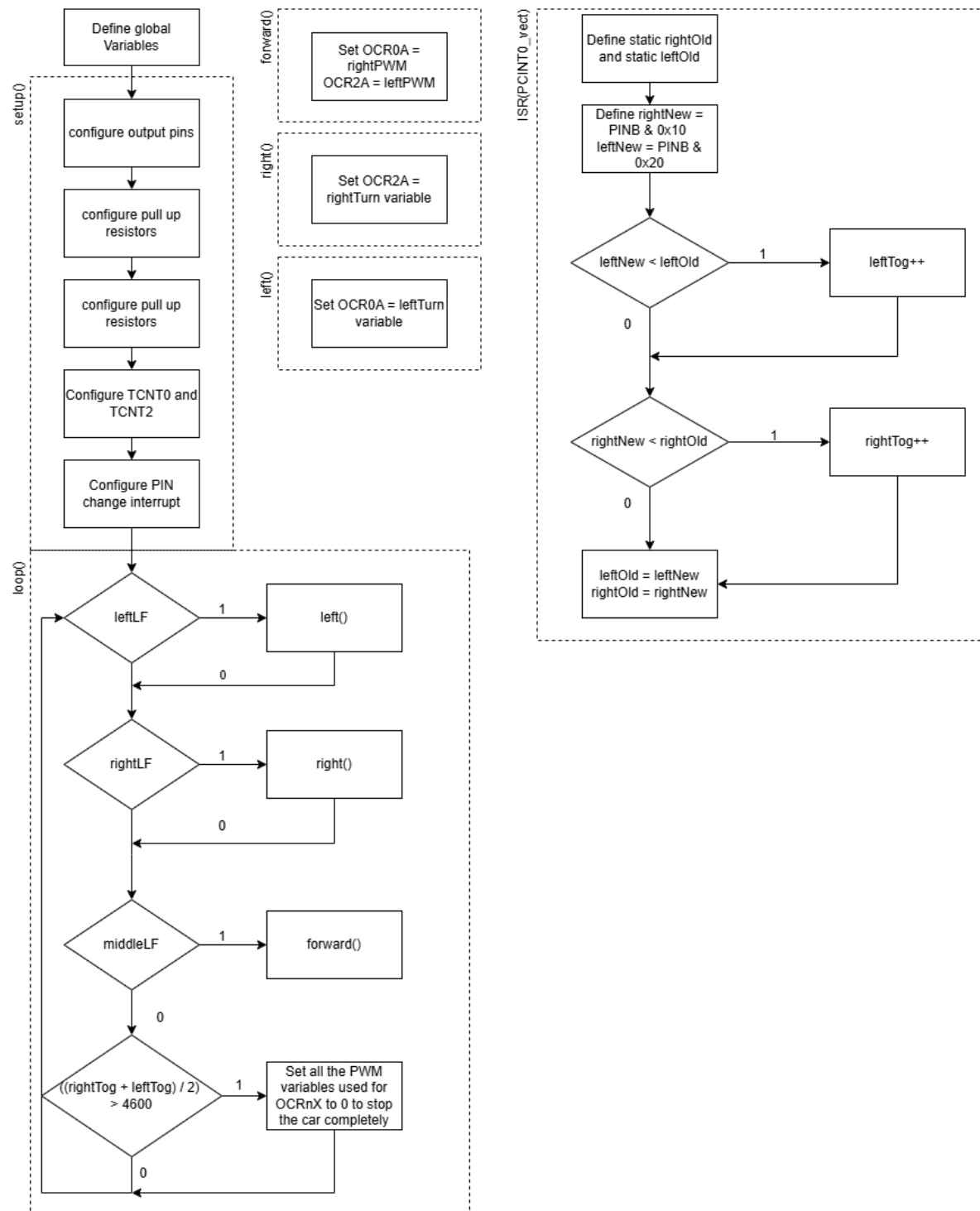
Flowchart

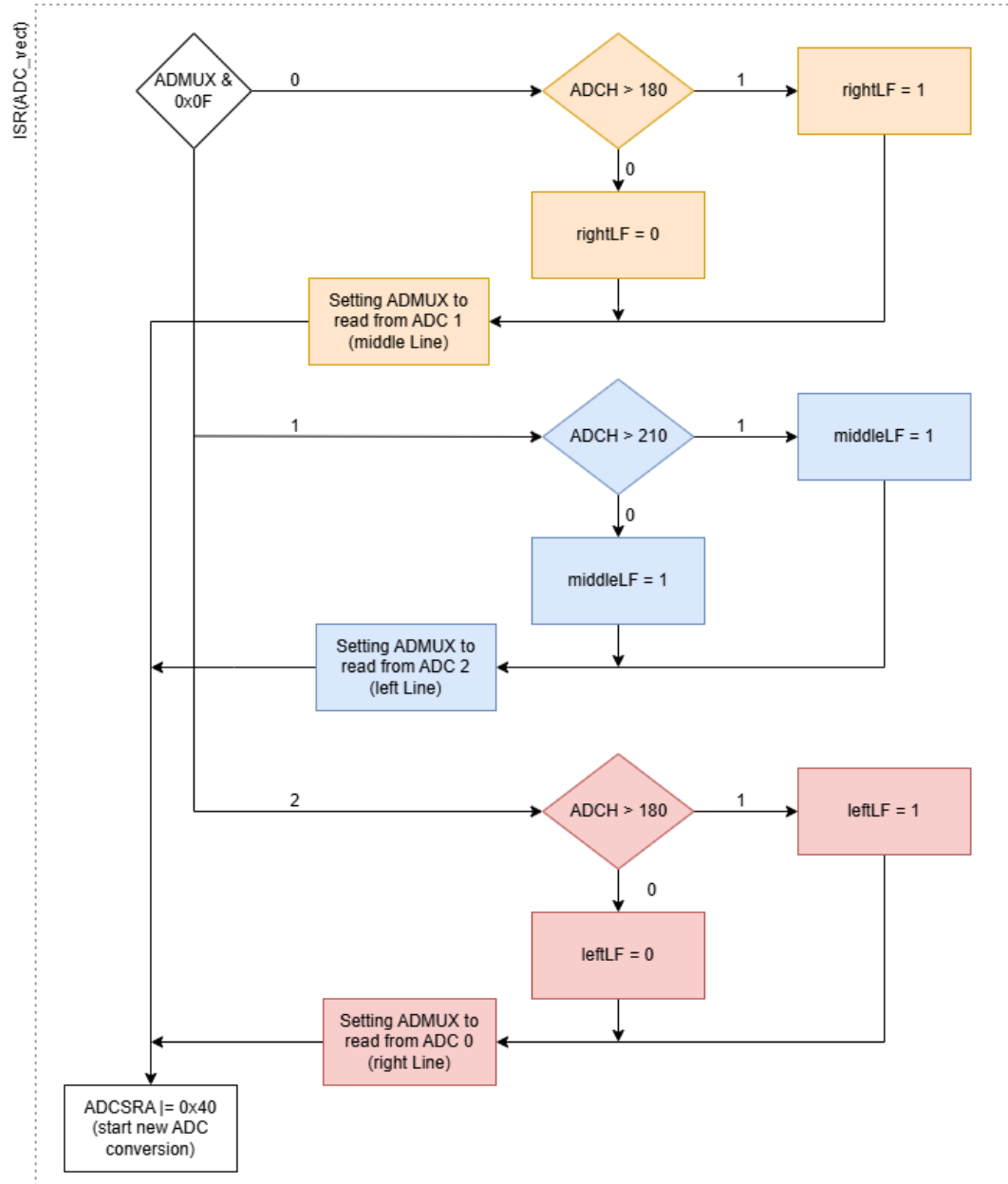
1) Barrier Detection





2) Line Following





I/O Pins Used

Barrier Detection:

A4: MUX control pin on Trigger Pins for Ultrasonic Sensors PC4
 D3: Left Motor Backward Speed OC2B (OUTPUT) PD3 -> Pin 7 on H-Bridge
 D5: Right Motor Backward Speed OC0A (OUTPUT) PD5 -> Pin 10 on H-Bridge
 D6: Right Motor Forward Speed OC0B (OUTPUT) PD6 -> Pin 15 on H-Bridge
 D8: Muxed Echo pin from Ultrasonic Sensor (INPUT) PB0
 D9: Front Ultrasonic Sensor Trigger Pin (OUTPUT) PB1
 D10: Right Ultrasonic Sensor Trigger Pin (OUTPUT) PB2
 D11: Left Motor Forward Speed OC2A (OUTPUT) PB3 -> Pin 2 on H-bridge
 D12: Left wheel encoder
 D13: Right wheel encoder

Line Following:

A0: Right line following sensor
 A1: Middle line following sensor
 A2: Left line following sensor
 D3: Left motor backward speed OC2B
 D5: Right motor backward speed OC0B
 D6: Right motor forward speed OC0A
 D11: Left motor forward speed OC2A
 D12: Left wheel encoder
 D13: Right wheel encoder

Hardware Description

For both Barrier Detection and Line Following:

Arduino:

We used an Arduino with the ATmega328P chip. It had current through the barrel battery (5V).

Wheel Encoders:

The wheel encoders have three connections, one connection to GND, one connection to VCC (5V), and one connection to Vin, which we chose as D12 for the left wheel encoder and D13 for the right wheel encoder.

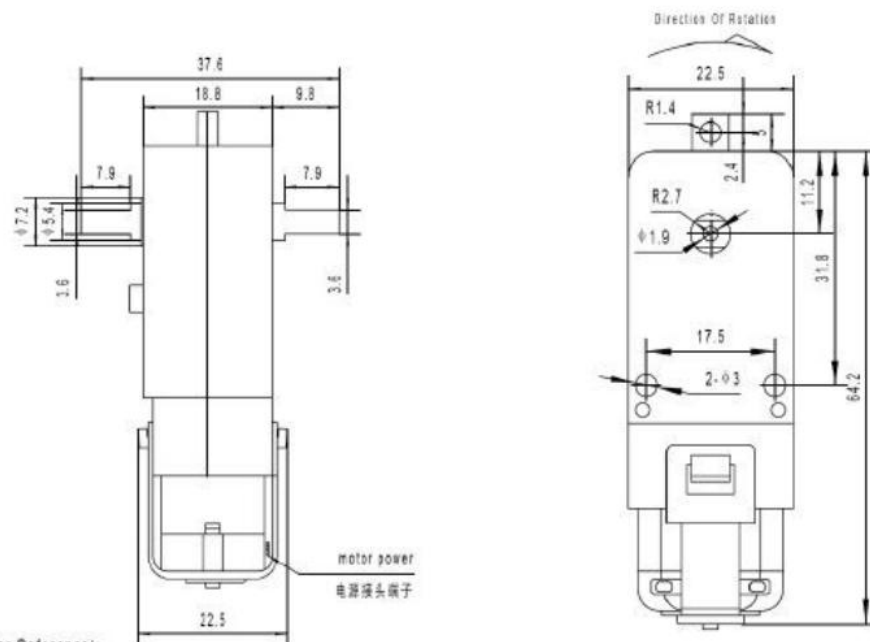
The wheel encoders work by using a magnet with 8 poles, toggling its state from HIGH to LOW 4 times per revolution, and we want to use these toggles to keep track of distance. So, we made sure in software to only add a toggle to our variables in a falling edge.

Keeping track of the toggles allows us to keep track of the distance our car has advanced and when it should stop.

Geared DC

Motors:

DG01D-A130GEARMOTOR



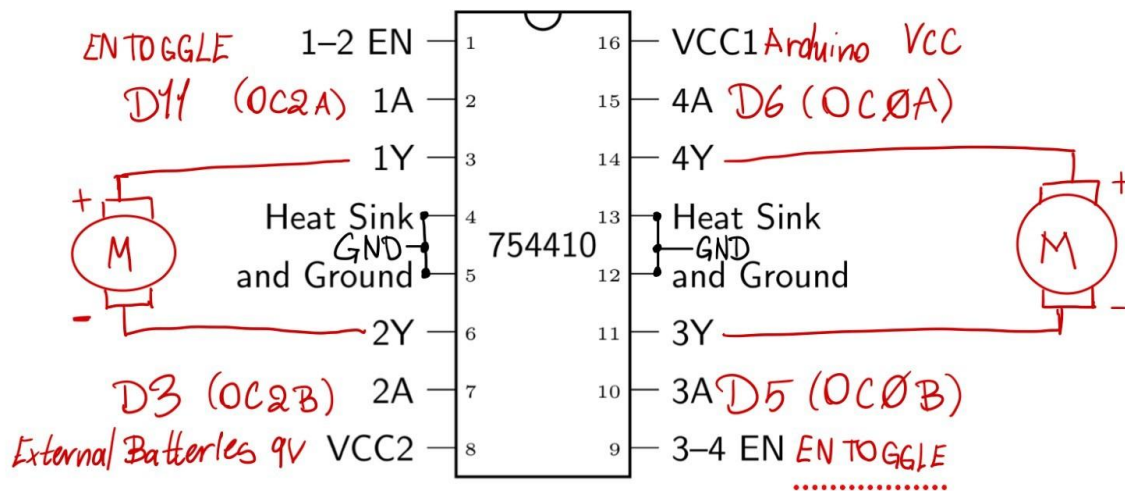
GEAR BOX Specification(For Reference):

- Suggested Voltage: 4.5V DC
- 1、No Load Speed: 90 ± 10 rpm
 - 2、No Load Current: 190mA (250mA MAX)
 - 3、Allowable max. Torque: 800gf.cm min

These DC motors have two poles. Positive and negative. To be able to move, one of them must be high while the other one must be low. Changing which pole has the high and low will determine the direction the motor will go in. These motors will move forward if you connect a HIGH signal to the positive pole, and a LOW signal to their negative pole. If both wheels are turning in the same direction, they will make the car go forward, and if they are turning in different directions, they will rotate right or left.

754410 H-Bridge Motor Driver:

754410 Quadruple Half H Driver



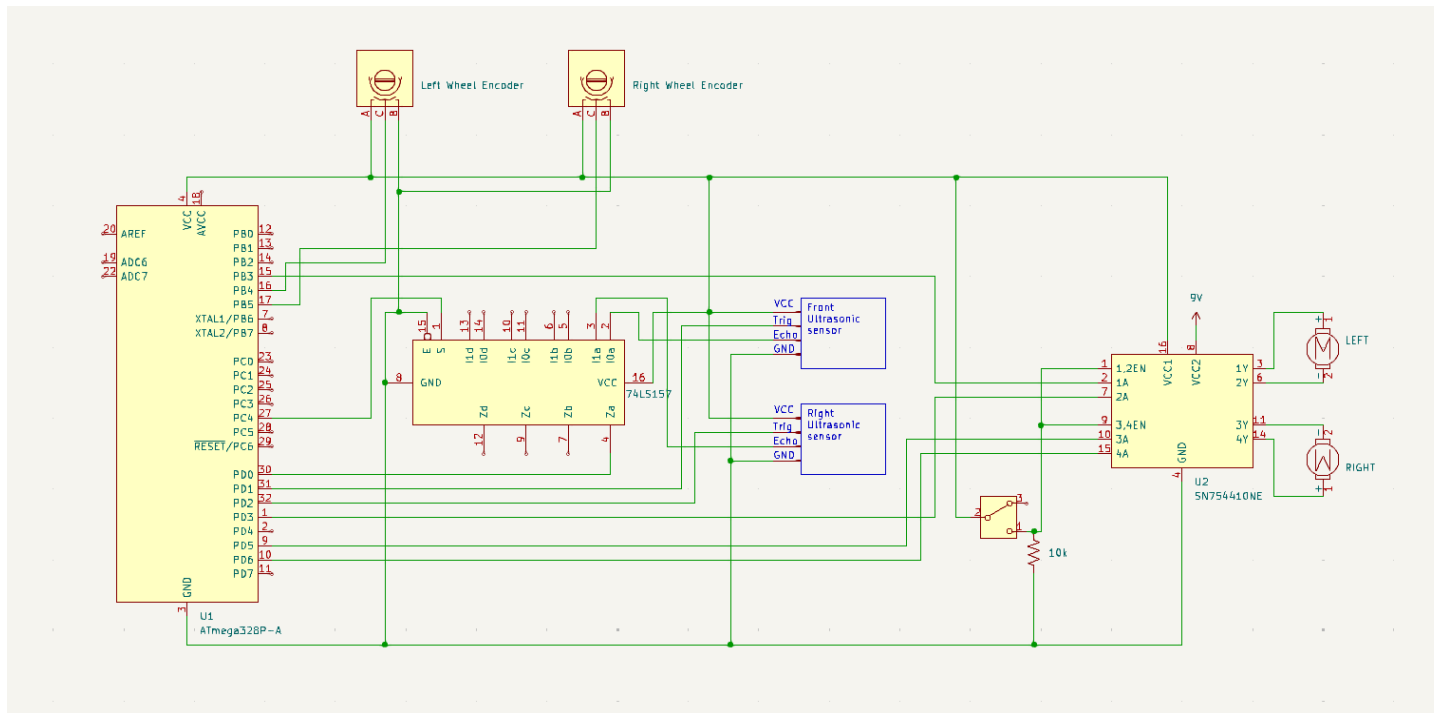
This chip allows us to control 2 DC motors at the same time. This chip allows us to use PWM in motors with a stronger power supply to make the motors go faster. In our case we used an 11V Lipo battery.

Connections:

1. 1-2 EN: Motor Enable Toggle switch connected externally
2. 1A: D11 (OC2A) which is the forward PWM signal for the left motor
3. 1Y: Positive pole of the left motor
4. GND
5. GND
6. 2Y: Negative pole of the left motor
7. 2A: D3 (OC2B) which is the backward PWM signal for the left motor
8. VCC2: External power supply, we connected a 9V battery
9. 3-4 EN: Motor Enable Toggle switch connected externally
10. 3A: D5 (OC0B) which is the backward PWM signal for the right motor
11. 3Y: Negative pole of the right motor

- 12. GND
- 13. GND
- 14. 4Y: Positive pole of the right motor
- 15. 4A: D6 (OC0A) which is the forward PWM signal for the right motor
- 16. VCC1: Arduino VCC

1) Barrier Detection:



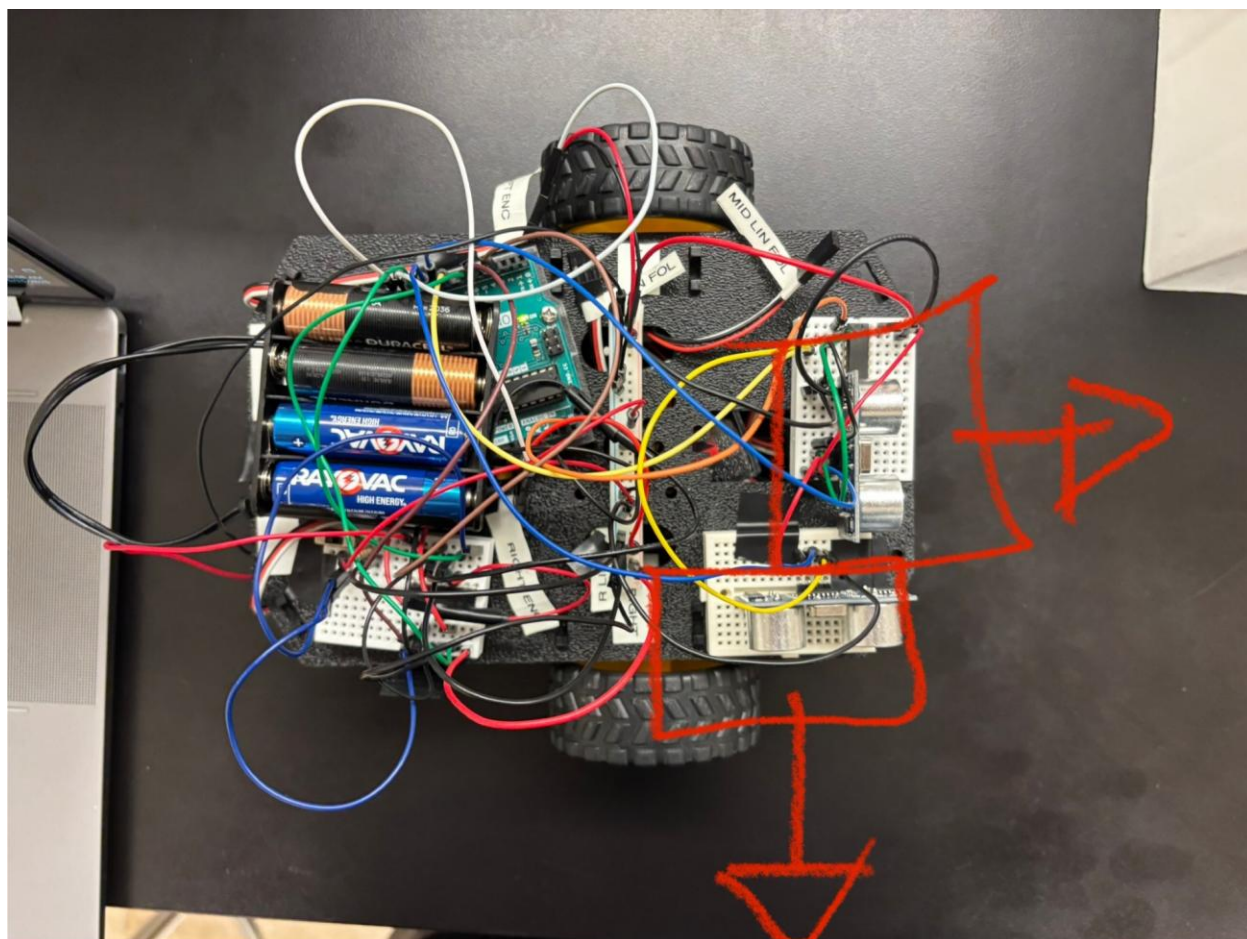
Ultrasonic



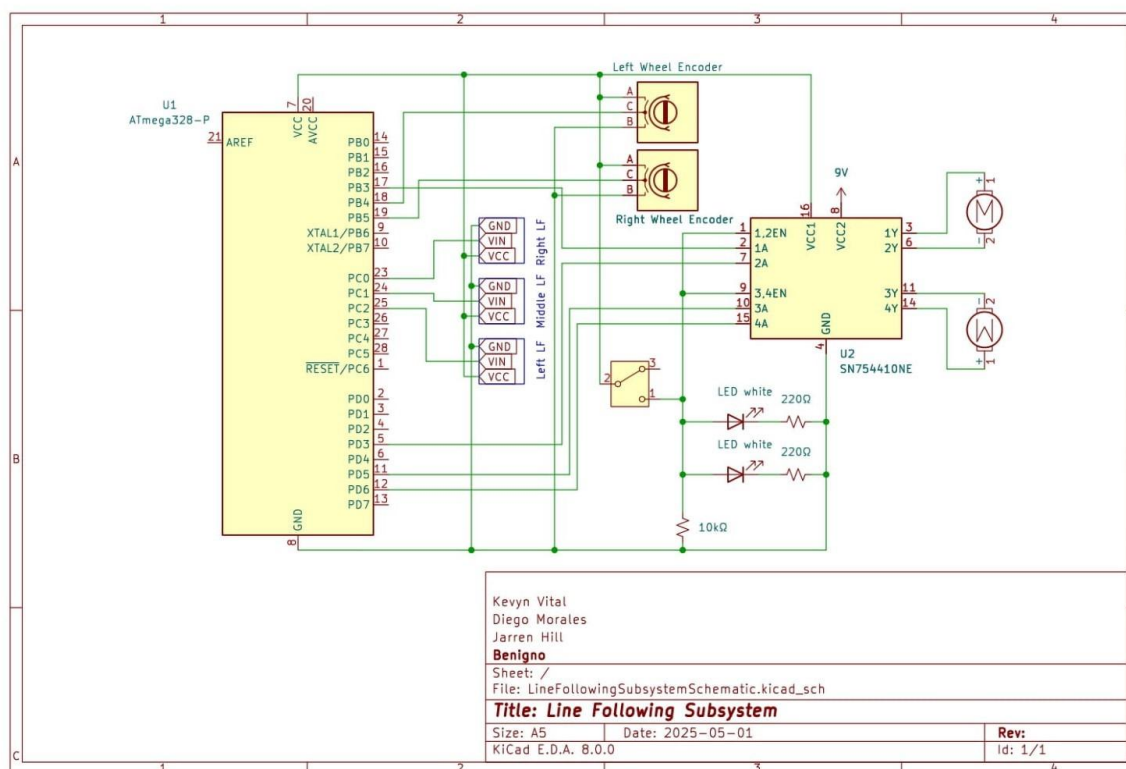
Sensor:

The HC-SR04 can determine the distance from an object if it is in the range from 2-400cm. It has 4 pins: VCC, Trig, Echo, and GND. If a 10 μ s pulse is sent to the trigger pin, the module will send an ultrasonic signal that will then be received with the Echo pin. Distance can be measured by calculating the time from the signal was sent to the trigger pin to when it was received by the Echo pin.

We used two of these. One place facing the front in the middle front part of the car, and one facing the right in the right front part of the car. As shown in the following picture, this is to be able to determine if there is an object in the front and on the right of the car.



2) Line Following



Line Followers:

The line followers have three connections. One connection to GND, one connection to VCC (5V), and one connection to Vin, which we chose as A0 for the right LF, A1 for the middle LF, and A2 for the right LF.

The line followers are mounted in front of the smart car, and they are close to the ground when the smart car is placed down. We utilize the line followers to determine if our smart car will drive forward (in the case that the middle sensor is on the black line), if the smart car will turn right until the middle sensor is centered on the black line (in the case that the right sensor is on the black line), and if the smart car will turn left until the middle sensor is centered on the black line (in the case that the left sensor is on the black line).

The way the line sensors work is by using infrared light. If they are over white, they will give a low ADC value, and if they are over black, they will give a high ADC value.

LEDs:

We found that by connecting two white LEDs and pointing them down in the direction of the line followers we were able to collect more consistent ADC data for the line follower values because

there was a greater contrast between the black line and the white board. The LEDs were connected to a switch that is the EN in the H-Bridge, so when the switch is turned off, the LED's along with the Arduino and the motors are turned off. These LED are connected to 220-ohm current limiting resistors as well.

Software Description

1) Barrier Detection

Global variables:

We have constants leftPWM and rightPWM that will store the values that will be written to the OCRxn registers to control the motor speeds. In addition to that, we have volatile unsigned long leftTog and rightTog, that will store the number of falling edge toggles of the left and right wheel encoders in an external interrupt. These help us measure distance moved in encoder ticks. Even though these values will be set to 0 whenever a wall is detected, we made them longs to avoid the risk of any overflow, using ints would have sufficed, however.

We defined two volatile unsigned long arrays captFront[2] and captRight[2] to store the input capture times (rising and falling edge) from the ultrasonic sensors. captFront[0] and captRight[0] store the rising edge timestamps, while captFront[1] and captRight[1] store the falling edge timestamps. The variable count is incremented by 65536 every time Timer1 overflows, giving us an accurate number of ticks from the start of the program until the last overflow happens.

We also had a volatile unsigned char named statusVar where we stored the value of SREG at the start of all interrupts and then restore the value at the end.

setup():

We set D3, D5, D6, D9, D10 and D11 as outputs for controlling motors and ultrasonic triggers. We set A4 as output for MUX control, enable internal pull-ups on D12 and D13 (wheel encoder inputs), configure TCNT0 and TCNT2 for phase correct PWM mode with prescaler of 1. TCNT1 is configured in normal mode with a prescaler of 256, the ICU is enabled with overflow interrupts. Pin change interrupts on D12 and D13 are also enabled for the encoders. ICNC1 is also set as well.

Explanation of how our forward(), right(), left(), and reverse() functions work:

For each of our motors, we have 2 poles. If one of them is high and the other one is low, it will turn a certain direction, if the poles are inversed it will rotate the other direction. We have two output compare PWM signals that are sent to each motor, OCnA and OCnB. If we set one of these to have a certain nonzero duty cycle and the other one to have a duty cycle of 0, the one with a duty cycle of 0 will always have a low signal, therefore making the motor able to rotate in a direction. In this circuit, applying nonzero values to OCR0A and OCR2A (forward signals) and zero to OCR0B and OCR2B (reverse signals) causes both motors to rotate forward. The values given to OCR0A and OCR2A are different, because both motors are not equally strong, so we first had to do trials to see the best value to make it go straight.

For the right motor, OCR0A is forward on D6, and OCR0B is reverse on D5. For the left motor, OCR2A is forward on D11, and OCR2B is reverse on D3. To move forward, we apply a nonzero PWM value to OCR0A and OCR2A (right and left forward) while setting OCR0B and OCR2B to zero. This makes it so that both motors rotate in the same direction and drive the car forward. Some of these functions will also be used with line following with different values written to them. A problem that we faced during the last maze demo was that our left wheel would not reverse. The solution for that problem is to completely turn off the OC2A output to avoid it sending a signal. Therefore, in the following functions OC2A will be turned off when the left wheel is reversing and turned back on when it is going forward. This is done with TCCR2A |= 0x80; In right(), OC2B is not used because the left motor rotates forward (OC2A enabled).

void forward():

This function sets the PWM of the 2 timers to go forward by setting the values that correspond to the right wheel:

- OCR0A = rightPWM (right motor forward)
- OCR0B = 0
- TCCR2A |= 0x80

The values that correspond to the left wheel:

- OCR2A = leftPWM (left motor forward)
- OCR2B = 0

void right():

- OCR0A = 0
- OCR0B = rightPWM (right motor reverse)
- OCR2A = leftPWM (left motor forward)
- OCR2B = 0
- TCCR2a = 0x80

void left():

- OCR0A = rightPWM (right motor forward)
- OCR0B = 0
- OCR2A = 0
- OCR2B = leftPWM (left motor reverse)
- TCCR2A = 0x20
- TCCR2A &= ~0x80

void reverse():

- OCR0A = 0
- OCR0B = rightPWM (right motor reverse)

- OCR2A = 0
- OCR2B = leftPWM (left motor reverse)
- TCCR2A = 0x20
- TCCR2A &= ~0x80

loop():

At the start of our loop, we start by sending a pulse of 10us to one of the two Ultrasonic Sensors. We determine which one is the one receiving the pulse by checking the value of pin A4, that is the control bit for a MUX of the right and front Echo signals from the Ultrasonic Sensors. If A4 is 1, it will send the pulse to the right Ultrasonic Sensor, else it will send it to the front one. We then will define two static unsigned long variables, rightDistance and frontDistance. These are static to avoid them being reassigned to 0 and messing up our logic forwards.

After that, it will check if `captRight[1] > captRight[0]`. If true, it will calculate the distance from the right Ultrasonic Sensor to the nearest obstacle by using the following equation: $\text{rightDistance} = ((\text{captRight}[1] - \text{captRight}[0]) * 340LL) / 125$. Giving the distance in mm. We do the same with the front sensor, if `captFront[1] > captFront[0]`, then $\text{frontDistance} = ((\text{captFront}[1] - \text{captFront}[0]) * 340LL) / 125$.

We then call the `forward()` function, and we check if the readings from the front sensor are greater than 85mm, if it is, then it will call `reverse()` and set `leftTog` and `rightTog` = 0. And create a while $((\text{leftTog} + \text{rightTog}) / 2) < 4$, it will keep reversing. We decided to use this strategy with the encoders as a delay instead of using the `_delay_ms()` functions, because no matter how fast the wheels are moving, they will always travel the same distance ideally. After that, it checks if there is an object detected with the right sensor at less than 170mm. If there is one, it will call `left()` and then set `leftTog` = 0 and we delay by waiting until `leftTog` reaches 27, which corresponds to a fixed rotation based on encoder ticks. If there was no object to the right, it would call `right()` and it will use the same delay strategy of setting `rightTog` = 0 and using a while loop.

ISR(PCINT0_vect):

This interrupt is triggered whenever there is a change in signal on the wheel encoder pins D12 and D13. Inside the interrupt, we compare the current value of each encoder pin to the previous value to check for a falling edge. We detect a falling edge if the previous value was high (1) and the current value is low (0). If a falling edge is detected on the left encoder pin, we increase `leftTog`, if it's on the right encoder pin, we increase `rightTog`. This lets us count how many times falling edge toggles the wheels have had.

ISR(TIMER1_OVF_vect):

Whenever there was an overflow of TCNT 1, this interrupt would add 65536 to the global variable count to keep track of all the ticks that have happened since the start of the program until the last overflow.

ISR(TIMER1_CAPT_vect):

This interrupt would be called whenever the input capture unit (D8) received a rising or falling edge signal by changing the ICES value every whenever the interrupt was called. And it will set our captRight or captFront array entries to the number of ticks from the start of the program until the rising or falling edge signal being captured.

We started by checking the value of ICES bit of the TCCR1B register by using if (TCCR1B & (1 << ICES1)). ICES1 represents bit 6 of TCCR1B. To check its current value, we use TCCR1B & 0x40.

If ICES is 1, that means it was receiving a rising edge signal. We then checked the MUX bit value A4. If A4 is 1, it would be receiving a rising edge signal from the right Ultrasonic Sensor, so we set captRight[0] = count + ICR1. Otherwise, it would be receiving data from the front sensor, so we set captFront[0] = count + ICR1. Regardless of the value of A4, it would clear the ICES1 pin to read a falling edge signal for the next time this interrupt is called.

If it was receiving a falling edge signal, it would check again the value of A4, if it was 1, it would set captRight[1] = count + ICR1, and then it would clear the value of A4 so that it would MUX to the front Ultrasonic Sensor the next time. If A4 is 0, it would set captFront[1] = count + ICR1, and then it would set the value of A4 so MUX to the right Ultrasonic Sensor. Regardless of the value of A4, we would then set the ICES1 pin to read a rising edge signal the next time.

2) Line Following

Global variables:

volatile unsigned char leftLF = 0, middleLF = 0, rightLF = 0 are declared global so that we can access them in the ISR for ADC interrupts. These variables act as flags that indicate whether a specific line sensor currently detects the line (receives an ADC value above a certain threshold). They are updated automatically inside the ADC interrupt and used in the loop() to decide the cars movement. Since the value is only 0 or 1 unsigned char is the datatype.

Below are the global variables that store the values given to OCRxn, they must be global so we can access them in our forward(), right(), and left() functions. Our left motor is stronger than the right motor, so we compensate by giving it a (145/144 ratio). Values are always positive and never exceed 255 so unsigned char.

- Volatile unsigned char rightPWM = 145

- Volatile unsigned char leftPWM = 144
- Volatile unsigned char turn = 125
- Volatile unsigned char turnSupport = 50

The turn PWM values must be lower than the PWM values for the forward segments because we do not want the car to aggressively turn and lose the black line.

The volatile unsigned char statusVar is used to save & restore SREG when entering and exiting our ISRs to protect our control flow.

void setup():

Our setup sets our left and right motor backward and forward speeds (D3, D5, D6, D11) as outputs, while keeping our right, middle and left line following sensors (A0, A1, A2), and left and right wheel encoders (D12, and D13) as inputs. We activate internal pull-up resistors for D12 and D13 to prevent unpredictable input readings on the wheel encoders. We set up the Pin Change interrupt for pins D12 and D13. We configure the Timer/Counter 0 and Timer/Counter 2 for phase correct with a prescaler of 1, no interrupts, and OC0A, OC0B, OC2A, and OC2B activated non-inverting mode. Finally, we set the ADC to auto-trigger disabled meaning we must manually start each conversion (we start our first conversion inside the setup). We also used cli() and sei() to turn interrupts off at the start of the setup and turned them back on at the end.

void loop():

The loop function determined the car's behavior based on the line follower sensor flags, which are updated in the ADC ISR. If leftLF is set to 1, meaning the left sensor has detected the tape, the car calls left() to turn left.

If not, it checks if rightLF is set to 1, if it is, right() is called to turn the car right.

If neither side sensor is active, but middleLF is set to 1, the car continues straight by calling forward(). If none of the line follower flags are set, the car will not update its movement, and it will continue its last command until it is stopped. After that, there is an if statement that will set the global PWM variables to 0 whenever the car reaches the distance necessary. When this happens, the car will stop even if there are any readings from the line following sensors.

void forward():

This function is the same as in the maze demo

- OCR0A = rightPWM (right motor forward)
- OCR0B = 0
- OCR2A = leftPWM (left motor forward)
- OCR2B = 0

void right():

- OCR0A = 0

- OCR0B = turnSupport (right motor reverse)
- OCR2A = turn (left motor forward)
- OCR2B = 0

void left():

- OCR0A = turn (right motor forward)
- OCR0B = 0
- OCR2A = 0
- OCR2B = turnSupport (left motor reverse)

ISR(PCINT0_vect):

This interrupt is exactly the same as in the maze demo. We keep track of the falling edge toggles of both wheel encoders.

ISR(ADC_vect):

This interrupt is called every time an ADC conversion finishes. We have a case switch where we read the last nibble from the ADMUX register and we read the value of one line following sensor each time the ISR executes, then we set the new conversion to read from the next line following sensor and so on.

First, we read the right sensor, if its value is above 180, we set the rightLF flag to 1, otherwise we clear it. Then we move to the middle sensor, and check if its value is above 210 to set middleLF flag. Last we read the left sensor, checking if it's above 180 to set leftLF. The reason that we have such a high value for middleLF is because we want the flag to be set when the middle line sensor is as centered as possible on the tape, which would reflect a bigger ADC value.

Peripherals

The ADC peripheral was used to use our line following sensors. The ADC was used with auto trigger disabled and ADC interrupts enabled, 128 prescaler, and in 8-bit precision mode. We had auto trigger disabled because we wanted to change between A0, A1, and A2 inputs from our three line following sensors. Therefore, inside of our ISR, we had to change where the next ADC value was going to read from and start a new conversion. The line following sensors are infrared sensors, and they give a higher value when they are facing black and a lower value if they are facing white.

We used the TCNT0 and TCNT2 for the right and left motor settings respectively. Both of them were set up with pulse-width modulation, in phase correct mode and a prescaler of 1.

TCNT1 is used by ultrasonic sensors. This timer/counter was set up to normal mode and TCNT1 is 16 bits, a 256 prescaler, and input capture enabled. In addition, it had input capture and overflow enabled. The input capture interrupt would be triggered every time a falling or rising edge signal was captured from D8 (we are able to capture both falling and rising edge by changing the setting of ICES1). And the overflow interrupt would be called whenever the counter got to the value of 65535, which takes about 1,048,576 us to overflow with these settings.

Our General-Purpose Inputs and Output pins were used for the wheel encoders by activating a PIN change interrupt for them. They were also used to enable the LCD select registers and transmit data.

Functionality

1) Barrier Detection

The smart car's barrier detection system is implemented using two ultrasonic sensors, one facing forward, and one facing right. They both work in tandem with wheel encoders and PWM motor control to navigate obstacles autonomously. The software begins by configuring PWM outputs on pins D3, D5, D6, and D11 using Timer/Counter 0 and Timer/Counter 2. These timers are set in phase-correct PWM mode with a prescaler of 1, giving precise speed control to the left and right DC motors connected to the H-Bridge.

The ultrasonics sensors have their trigger pins connected to digital outputs on D9 (front) and D10 (right). Echo signal received with the ICU on pin D8, with the analog A4 acting as a multiplexer control pin to alternate between sensors. Timer1 is used in normal mode with a prescaler of 256 to capture the echo pulses with rising and falling edges, which are handled in the `TIMER1_CAPT_vect` ISR. The time in between the falling and rising edges is converted into distances in the main loop.

Wheel encoders are connected to D12 and D13 to give us feedback on motor movement by detecting falling edges with pin change interrupts. The number of toggles from each encoder is tracked by the `PCINT0_vect` ISR, which allows the car to measure distances traveled using maneuvers, and it allows us to have precision with our turns or reversing the car no matter how fast the motors are moving.

The car moves forward by default, if the front ultrasonic sensor detects a barrier within 85 mm, the car reverses briefly, then checks the right ultrasonic sensor. If the right sensor also detects a wall within 170 mm, the car performs a left turn. If the right path is clear, it performs a right turn instead. Each maneuver is timed using our wheel encoder thresholds based on distance, ensuring that the car has consistent turn angles. The consistent turn angles made it possible for us to position the smart car in a way that it would only have to take four or five turns before reaching the end of the maze, if placed in the correct orientation. We successfully completed the final barrier demo in 12.68 seconds.

2) Line Following

The smart car's line detection and speed subsystem works by using the ADC to read the three line-following sensors connected to A0 (right), A1 (middle), and A2 (left). Each sensor outputs an analog voltage based on the surface it detects. These values are compared to fixed thresholds in the ADC ISR. If the sensor detects black tape, its flag (`rightLF`, `middleLF`, or `leftLF`) is set to 1. If it detects the white background, the flag is set to 0.

Inside the loop function, the car checks the sensor flag and selects a direction. If the left sensor detects the line, the car turns left by calling `left()`. If not, and the right sensor detects the line, the car turns right using the `right()`. If neither the left and right sensors are over the line, and if the middle sensor is on the black line, it will go forward by using `forward()` function. If no sensor flags are asserted, the car keeps executing the last movement until one of the flags is updated again by the ADC interrupt.

The smart car's motors are controlled by OCR0A, OCR0B, OCR2A, and OCR2B. These control the two H-bridge motor driver channels. The timers are set to phase correct PWM, which allows us to control both speed and direction carefully.

To move forward, OCR0A and OCR2A (forward pins on D6 and D11) are set to a nonzero value, both motors strengths are different so these values have to be calibrated to actually go forward, in addition of OCR0B and OCR2B being set to 0. For turning, we have one of the motors rotate forward with a faster speed and the other one move backwards with a slower speed. The turning PWM values are smaller than the forward PWM values to ensure smooth turns that help the car stay centered on the line without overcorrecting.

To measure distance travelled, we use two wheel encoders connected to D12 and D13. Each encoder's falling edge signal is monitored through a pin change interrupt. In the ISR, we compare the current state of the encoder pin to its previous state. A falling edge increments `leftTog` or `rightTog`, keeping count of how far each wheel has rotated. Once the average of the encoder toggle count exceeds a value of 4600, the car stops automatically by setting all of the PWM values (`rightPWM`, `leftPWM`, `turn`, and `turnSupport`) to zero.

This demo did not meet the final line-following track requirement. The car follows the line accurately using the sensors but is inconsistent in stopping on its own after reaching the target distance. Our smart car had trouble counting the same number of toggles each time it went through the track, and we were unable to complete the final demo in the time given to us. This happened because our car would sometimes take wider or tighter turns when doing the track, making the number of toggles needed inconsistent and unpredictable.

Design Changes

1) Barrier Detection

Initially, our barrier detection strategy relied on a whisker-based system. The logic was: when a whisker was triggered, the car would turn 90 degrees. If the same whisker was triggered again, the car would instead turn 180 degrees, allowing it to make both right and left turns depending on the environment. This method required the car to maintain a precise orientation throughout the maze:

Later, we decided to try using ultrasonic sensors instead of whiskers. We mounted one sensor facing forward and another to the right side of the car. Our strategy was to always follow the right wall.

- If the right sensor stopped detecting a wall the car would turn right
- If the front sensor detected a wall while the right still sensed one, the car would turn left

We encountered hardware issues with the motors during the barrier detection demonstration, and once fixed, we realized a big flaw in our approach as some of the sections of the maze were too narrow for the car to navigate while staying perfectly parallel to the wall. This made it so that our precision dependent strategy was impractical.

We revised our approach, and instead of using strict 90 or 180 degree turns, the car would perform small adjustments after sensing a wall with the front Ultrasonic Sensor:

- If the left ultrasonic sensor detected a wall, the car would turn right slightly
- If the right sensor detected a wall, it would turn left slightly

This is explained in further detail in our software description and functionality.

2) Line Following

After the initial line detection demo, since our rightTurn and leftTurn values were so similar, we switched to one variable “turn” which would be the main PWM value for the direction that we want to go in during a turn. If we want to turn right, the left wheel will receive the turn PWM value, and the right wheel will receive our newly implemented turnSupport PWM value, which allows for smoother turning, as we noticed sharp turns made our car drift and lose track of the line more often.

After receiving feedback from Dr. P, we found that it was bad practice to use a dummy variable to check a condition when we could actually be probing the value of importance. Instead of doing `switch choice = 1, then switch (choice)`, we do `switch(ADMUX & 0x0F)` in our refined code.

During the last week, we were trying to implement Proportional control to our Line Following logic. This way we would be able to efficiently go through the maze. Our thought was just to add Proportional control to our forward() function, and so being able to have the middle sensor stay on the middle line for longer. However, we were not able to get it going by the end of Thursday before class, so we decided that we should stick to not using any proportional control. The ideas we had and why they didn't work will be explained in the next section.

The final adjustment we made was to the amount of wheel encoder toggles before the car stops. We found that approximately 4600 toggles is the amount the smart car needs to read to stop around the end of the line.

Feedback Control

We attempted to implement Proportional control in our smart car's line following subsystem. Our goal was to adjust the motor speeds while the car was driving straight, helping it maintain centered alignment on the black tape for more accurate line following. The idea was to only apply proportional feedback when the middle line sensor A1 detected the black line and use fixed turn behavior otherwise. The control logic was integrated into our main loop: whenever middleLF was set (the car was centered on the line), we adjusted the global rightPWM and leftPWM values using a proportional control based on the error between a target ADC value and the actual reading from the middle sensor.

We had setpoint (sP) = 230, which was the typical ADC value we received when the middle sensor was perfectly aligned with the black line.

Process variable (pV) = ADCH, the most recent ADC reading from the middle line sensor

Error (err) = sP - pV, and Kp = 2 which we decided on after trial and error with overcorrection and not correcting fast enough.

rightPWM -= (Kp * err) / 10;

leftPWM += (Kp * err) / 10;

This meant that if the middle sensor was deviating from the center, the PWM signals would be adjusted proportionally, making the car correct itself by subtly speeding up with one wheel while slowing the other.

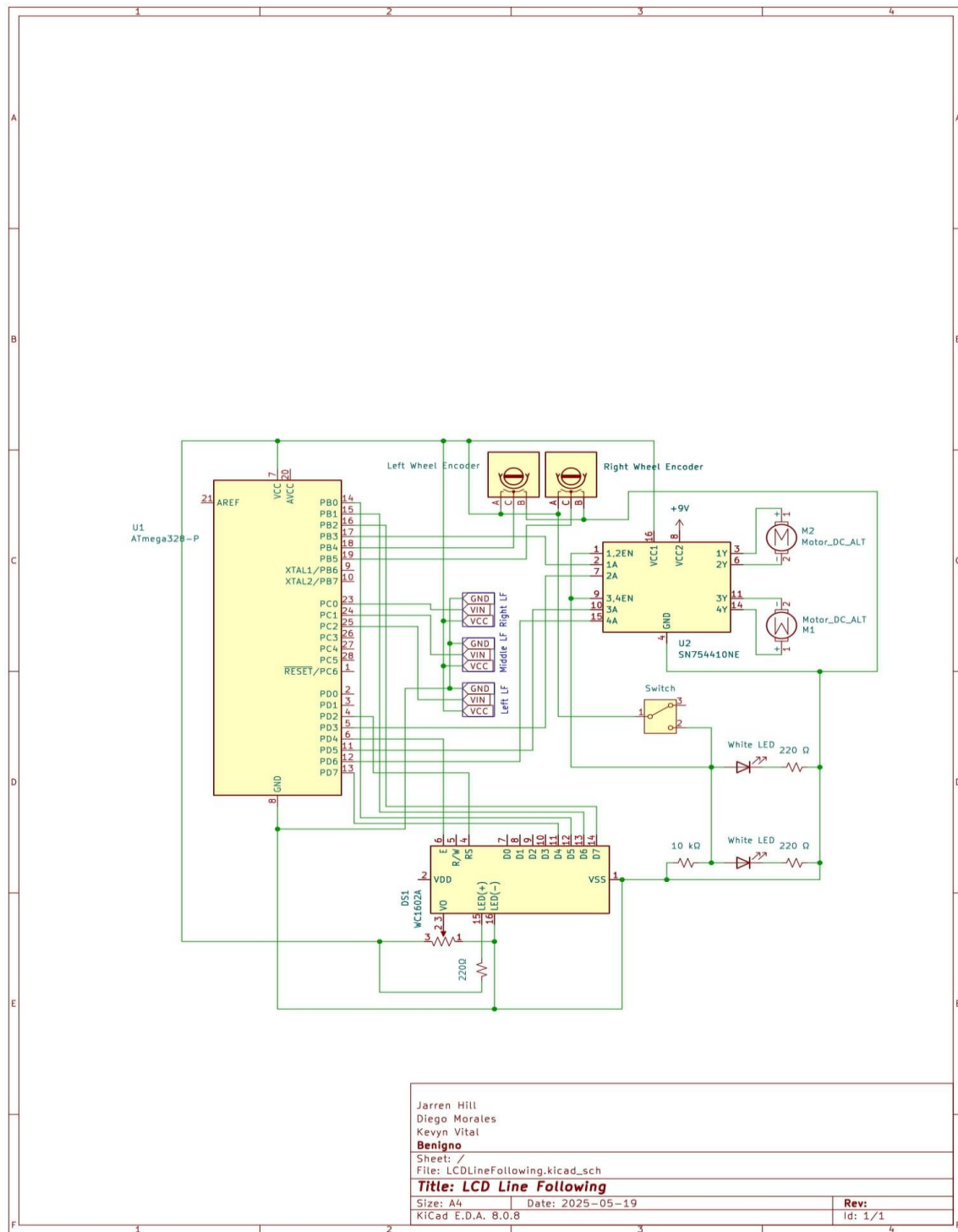
This implementation did not produce a desired result. The main issue we had was that the car could not determine the correct direction of correction. Without context from the side sensors (leftLF and rightLF), the sign of the error alone wasn't enough to indicate whether the car had veered left or right. As a result, the car would always adjust itself to veer to the right, because it

would always decrease the rightPWM value and increase the leftPWM value whenever it wasn't in the completely center of the line.

To improve this system, we realized that we also needed some type of proportional term from the right and left line following sensors. Probably trying to have the right and left line followers always be on white on top of trying to have middle over black all the time. This way we could ensure that it would veer in the right direction. In addition, with additional time, we would have also wanted to implement PI control to improve the car even further.

Even though we were not able to correctly implement proportional control into our smart car demo, the process gave us insight into the challenges of implementing feedback control into the smart car.

Enhancement 1 – LCD Line Following



Our first enhancement is an odometer display on an HD44780 LCD screen, which displays how far the smart car has traveled during the line following demo in cm.

Instead of just following a line, the smart car now also uses wheel encoders to measure distance and display it in centimeters on the LCD.

Software Explanation:

We use very similar code to our line following subsystem. In our setup, we initialize the lcd using `lcd_init()`.

We are using our encoders to measure distance. And doing some calculations we can determine that the distance in mm is equal to the $(106 * \text{average toggles}) / 100$. So, we can rewrite the equation to be $\text{distance} = ((\text{rightTog} + \text{leftTog}) / * 106) / 200$. We got this equation from activity 10.

After we determine our distance in mm, we want to display it in cm with tenth's precision place. So, we create 2 arrays to store the distance in cm and with 11 entries (because the maximum possible value that we can get is about 4500000000, which is bigger than a long so it would never reach that). The maximum number possible has 10 digits + null character. And the second array has 2 entries, because it is just the decimal place which is 1 digit + null character. We used `ltoa` to store the distance in cm in the array for it and `itoa` for the decimal place.

After that, we displayed "Odometer:", went to the second line, then the distance in cm, '.', and the decimal place, and "cm" after that.

The output would look like:

Odometer:

12.3cm

The other logic from the line following trial was still there but it was previously explained in other sections.

We are not sure the reason why this did not work the day of the demo, and we did not have enough time to keep changing things to make it work.

Enhancement 2 – Three Point Turn

(same hardware setup as Line Following)

After the car drives forward and accumulates approximately 200 encoder toggles, it will automatically perform a three-point turn without any external trigger.

The goal is to place the car almost at the end of the track so that it can travel the 200 encoder toggles and then at the end, perform a three-point turn and go back to the track. This three-point turn would bring it to the original spot where it started it but facing the opposite direction.

Software Explanation:

Inside the loop, we have a condition that checks whether the car has moved an average of 200 toggles. Once this threshold is met, the car executed the `threePoint()` function.

The three-point turn works in three steps:

1. Initial turn right: The car turns right using `right()` until the left wheel accumulates 90 toggles using a while loop. This is to create a delay that allows the car to turn for long enough to face the right.
2. Reverse: The car drives backward using `reverse()` until the average of both wheels reaches 150 toggles.
3. Final turn right: The car turns right again using `right()` until the left wheel accumulates 90 toggles again. Both right turns should have the same amount of turn to be able to work and finish at the same place it started.

These toggles are detected by using pin change interrupts on D12 (left encoder) and D13 (right encoder), in the ISR, we count falling edges to increment `leftTog` and `rightTog`.

The reason this enhancement did not work the day of the trial was because the left wheel stopped going backward again, and therefore the car could not reverse. Unfortunately, we tried this only with a little time on our hands before the end of the class, so we were not able to fix the problem.

Challenges

Our team faced several unexpected challenges, which stem from hardware instability, inconsistent power source voltages, and code management issues.

Hardware reliability was a recurring obstacle, especially due to wire connections. Because the smart car involved many wires and movement, loose wires became a major source of failure. On the final day of testing, the car successfully navigated the line but unexpectedly stopped responding. After a significant time spent debugging the car, we discovered that a VCC connection for the line sensors had come loose, which is something we had not encountered before. This showed us a major vulnerability in our wiring layout. If we were to redo the project, we would focus on putting more emphasis on clean wire management and using more cables cut to length for important components of the smart car. In addition to that, at one point our left motor stopped working and we were trying to debug it for a long time before figuring out that that was the problem. After that, we changed the motor and kept working on the car.

Power issues also gave us major setbacks. Earlier in the semester, we began experiencing inconsistent performance due to voltage drops. We tried to mitigate this with reusable 9V batteries, but they quickly lost effective charge after initial use, which left the car underpowered. To adapt, we resorted to powering the Arduino from a laptop or using spare AA battery packs for our final runs. If we could go back, we would have powered the Arduino from the laptop and saved the AA battery packs/9V batteries for the final demonstration days.

Another challenge was code management during the final barrier detection demo. Each member of the team developed and modified separate versions of the code, leading to confusion about which version was the most up to date or which was used for which test. Although we had a shared code repository, the lack of consistent file naming or documentation made it time consuming to trace functionality. In future projects, we would enforce a naming convention, add unique comment headers, and track changes more closely through GitHub to ensure better organization.

For functionality related challenges, we encountered motor issues during early barrier detection trials, our left wheel could not move backward, this wheel was connected to OC2A and OC2B. Due to this reason, we lost most of our time on the Monday Maze Demo. On that day, we resolved the problem by changing to TCNT1 instead of TCNT2, but that led us to change our strategy from using Ultrasonic Sensors to using whiskers again. And we did not have enough time to get it on Monday. After that day, Dr P told us that we could try turning OC2A completely off when going backwards, and it worked. So we went back again to use the ultrasonic sensor. But once we fixed that, we faced a bigger flaw in our design. The ultrasonic based navigation strategy relied on maintaining a precise parallel orientation to the walls, but this was difficult in narrow sections of the maze. Even minor misalignments caused us to get stuck, and in turn made us fail to clear the maze, so we found this was not a practical method if we had been given

multiple mazes to complete. If we were to implement a fix, we would add adaptive correction like watchdog timers.

Finally, our line following demo was only partially successful, although the car was able to follow the path accurately it was inconsistent at stopping at the correct endpoint. Our stopping logic was based on wheel encoder toggles, but these varied slightly with each run, possibly due to slippage. We were close, but unable to resolve this fully in the time allotted. Given more time, and less hardware issues, we could have tweaked the number of toggles before stopping to account for inconsistencies.

Conclusion

This project explored our smart car's development and the process behind both the barrier detection and line following challenges. We successfully implemented hardware systems using ultrasonic sensors, line followers, wheel encoders, and PWM motor control with an H-Bridge. Our software used timers, interrupts, and sensor-based decision making so that we could complete both of the demos.

The barrier detection demo was fully completed, navigating the maze in 12.68 seconds using ultrasonic feedback and encoder toggles for the turns. While the line following demo worked well in detecting and tracking the path, the car was inconsistent in stopping at the target distance because of variations in our toggle counts during the demo.

We experimented with P control and added enhancements like an LCD odometer and a three-point turn routine. While we did not use P control in the final demo or get to present our LCD odometer and three-point routine, it gave us insight on the challenges associated with these tasks.

We faced several obstacles like hardware instability, power issues, and trouble staying organized, but these experiences helped us better understand the importance of keeping track of all variables especially when working on a big project like this.